

Jackfruit Phase-2

{Project Plan Document}

Project: Student Attendance Management System (SAMS)

Subject Code: UE24CS341A — Software Engineering

Version: 1.1

Date: 16-09-2026

Status: Draft — For Submission

Project Team

USN	Name
PES2UG24CS350	Piyush A Patel
PES2UG24CS310	Neel Chandrakar
PES2UG24CS331	Nookala Sai Tushar Krishna
PES2UG24CS364	Pranav Somashekar

1. Introduction

1.1 Purpose

This document is the Jackfruit Phase-2 Project Plan for the Student Attendance Management System (SAMS), building on the Phase-1 Software Requirements Specification. It defines the development lifecycle, tooling, deliverables, work breakdown structure, team roles, effort estimate, testing strategy, and schedule that will guide the team from design through to the final demo.

1.2 Scope

This plan covers the planning and execution approach for all functional and non-functional requirements defined in the SRS (SAMS-F-001 to SAMS-F-040, SAMS-NF-001 to SAMS-NF-006, and SAMS-SR-001 to SAMS-SR-005). It does not itself specify detailed test case steps — those are covered in the Phase-3 Test Plan — but does establish the testing strategy, test-ID conventions, and testing schedule (Section 8).

1.3 References

- Software Requirements Specification (SRS) — Student Attendance Management System, Version 1.0
- Jackfruit Mini-Project Guidelines, Software Engineering (UE24CS341A), 2026

1.4 Definitions and Acronyms

- SAMS — Student Attendance Management System
- WBS — Work Breakdown Structure
- PM — Person-Month(s)
- UT / IT / ST — Unit Test / Integration Test / System Test
- RTM — Requirements Traceability Matrix

2. Software Development Lifecycle Model

The team will follow the Agile (Scrum) model, structured as seven 2-week sprints across the semester, as mandated by the course guidelines ("Agile methodology only").

Justification

- Requirements for an academic mini-project are refined iteratively as the course syllabus is covered week by week, making a plan-driven model (e.g. Waterfall) impractical.
- A four-person team with clearly separable modules (Authentication, Master Data, Attendance Engine, Analytics/Dashboards — see Section 6) benefits from Scrum's parallel sprint backlogs, letting each member work independently while integrating every sprint.
- Regular sprint reviews align naturally with weekly faculty evaluations and let the team demonstrate incremental working software rather than a single big-bang delivery.
- Short sprints surface integration issues (e.g., between the Attendance Engine and Analytics modules) early, reducing risk before the final demo.

3. Tools & Infrastructure

Category	Tool	Purpose
Planning & Task Tracking	GitHub Projects (Kanban board)	Sprint planning, backlog grooming, user stories
Design / Diagramming	draw.io, Graphviz	ER diagrams, UML use-case diagrams, architecture sketches
Version Control	Git + GitHub	Source control, feature branches, pull-request reviews
Development Environment	VS Code, Python 3.x, Django	Core application implementation
Bug Tracking	GitHub Issues	Logging, triaging, and resolving defects
Testing	Django TestCase / pytest	Unit and integration testing
CI Pipeline	GitHub Actions	Automated test runs on push / pull request
Documentation	Markdown, Google Docs / Word	SRS, README, project plan, test plan

4. Deliverables (Reuse vs. Build)

Deliverables are categorised below as Reuse (leveraging Django/framework-provided capability) or Build (custom logic specific to SAMS), to focus development effort where it adds the most value.

Deliverable	Category	Justification
Django framework & built-in auth	Reuse	Tested, secure session/authentication mechanics; avoids reinventing security-critical code
Django ORM & admin scaffolding	Reuse	Speeds up baseline CRUD scaffolding for master data entities
SQLite database engine	Reuse	Bundled with Django; sufficient for the project's academic scale
CSS styling (Bootstrap or similar)	Reuse	Consistent, professional styling without custom framework development
Shared base HTML template & navigation	Build	Project-specific layout so all four members' pages stay visually consistent
Authentication & RBAC logic	Build	Role-based access rules (Admin/Faculty/Student) are specific to SAMS
Attendance marking & editing engine	Build	Core differentiator; roster selection, present/absent capture, edit-window logic
Attendance calculation engine	Build	Subject-wise/overall percentage logic is a custom business rule
Low-attendance monitoring & alerts	Build	Configurable-threshold logic specific to SAMS' requirements
Reporting module	Build	Role-scoped report views tailored to Admin/Faculty/Student needs

Deliverable	Category	Justification
Dashboards	Build	Role-specific summaries unique to SAMS' three user types

5. Work Breakdown Structure (WBS)

The WBS decomposes SAMS into implementation-level tasks, each mapped to the functional requirements defined in the Phase-1 SRS (Section 4) and to the responsible team member.

WBS ID	Task	Sub-tasks	Linked SRS IDs	Owner
1.0	Project Setup & Environment Configuration	Repo init, Django project scaffold, base template, CI setup	—	All
2.0	Database Design	ER modelling, Django model classes, migrations	Appendix A	All (Neel lead)
3.0	Authentication & Security Module	Login/logout, session mgmt, password hashing, CSRF, server-side RBAC	SAMS-F-001–008 ,SAMS-SR-001–004	Tushar
4.0	Master Data Management	Student/Faculty/Section/Subject CRUD, faculty assignment, admin UI	SAMS-F-009–021	Neel
5.0	Attendance Marking Engine	Roster selection UI, mark Present/Absent, duplicate prevention, edit window	SAMS-F-022–027	Piyush
6.0	Attendance Calculation & Analytics	Subject-wise/overall % calculation, low-attendance threshold logic	SAMS-F-028–030 ,SAMS-F-035–037	Pranav
7.0	Reporting Module	Subject/section/student reports, attendance history views	SAMS-F-031–034	Pranav
8.0	Dashboards	Admin, Faculty, and Student dashboard views	SAMS-F-038–040	Pranav
9.0	Integration & System Testing	Cross-module integration, unit/integration/system test execution	—	All
10.0	Documentation Finalisation	SRS updates, README, user guide, Phase-3 Test Plan handoff	—	All
11.0	Final Demo Preparation	Demo script, working build, 15-min presentation	—	All

6. Team Roles & Responsibilities

Each member owns a distinct module end-to-end (design, implementation, unit testing) while contributing to shared activities like integration testing and documentation.

6.1 Roles Overview

Role	Team Member	Primary Responsibility
Lead Developer — Security & Auth	Nookala Sai Tushar Krishna	Authentication/RBAC design & implementation, security unit tests
Lead Developer — Master Data	Neel Chandrakar	Admin CRUD modules, database schema coordination
Lead Developer — Attendance Engine	Piyush A Patel	Attendance marking/editing engine, integration test coordination
Lead Developer — Analytics & Reporting	Pranav Somashekar	Calculation engine, reports, dashboards, final test report consolidation

6.2 Division of Work by Area

Work Area	Primary Owner	Supporting Member(s)
Database design & schema	Neel	All (review)
Authentication & security	Tushar	Piyush (integration into attendance views)
Master data management	Neel	Tushar (RBAC hooks on admin pages)
Attendance marking engine	Piyush	Neel (data model), Pranav (test data seeding)
Analytics, reporting & dashboards	Pranav	Piyush (attendance data source)
Integration & system testing	All	—
Documentation (SRS, README, plans)	All	—

7. Effort Estimation

Effort is estimated in person-months (PM), assuming part-time contribution alongside other coursework over a 15-week semester. Figures are rough estimates for planning purposes and will be refined during sprint planning.

WBS ID	Task	Effort (Person-Months)	Primary Owner
1.0	Project Setup & Environment Configuration	0.25	All
2.0	Database Design	0.50	All
3.0	Authentication & Security Module	1.00	Tushar
4.0	Master Data Management	1.50	Neel
5.0	Attendance Marking Engine	1.25	Piyush
6.0	Attendance Calculation & Analytics	0.75	Pranav
7.0	Reporting Module	0.75	Pranav
8.0	Dashboards	0.50	Pranav

WBS ID	Task	Effort (Person-Months)	Primary Owner
9.0	Integration & System Testing	0.50	All
10.0	Documentation Finalisation	0.25	All
11.0	Final Demo Preparation	0.25	All
	Total	7.50 PM	

8. Testing Strategy Preview & Schedule

This section previews the testing approach that will be fully detailed in the Phase-3 Test Plan: the levels of testing, the test-ID convention that keeps this plan consistent with the SRS, and the schedule for executing tests alongside development.

8.1 Test Levels

- Unit Testing (UT) — verifies individual functions/validation rules within a single module (e.g. duplicate-USN rejection, percentage-calculation formula), written by each module's owner as it is built.
- Integration Testing (IT) — verifies connected components work together (e.g. a marked attendance record correctly feeding the calculation engine).
- System Testing (ST) — verifies complete end-to-end workflows across roles (e.g. Admin sets up a section → Faculty marks attendance → Student views updated percentage).
- Security Testing (SEC) — verifies the security requirements SAMS-SR-001 to SAMS-SR-005 (password hashing, session enforcement, server-side RBAC, CSRF, SQL-injection prevention).

8.2 Sample Test Case Preview

A representative sample of the SRS-to-test-case mapping is shown below to fix the ID convention (TC-<Module>-##) ahead of Phase-3; the full test case specifications (steps, test data, expected results) will be authored in the Phase-3 Test Plan.

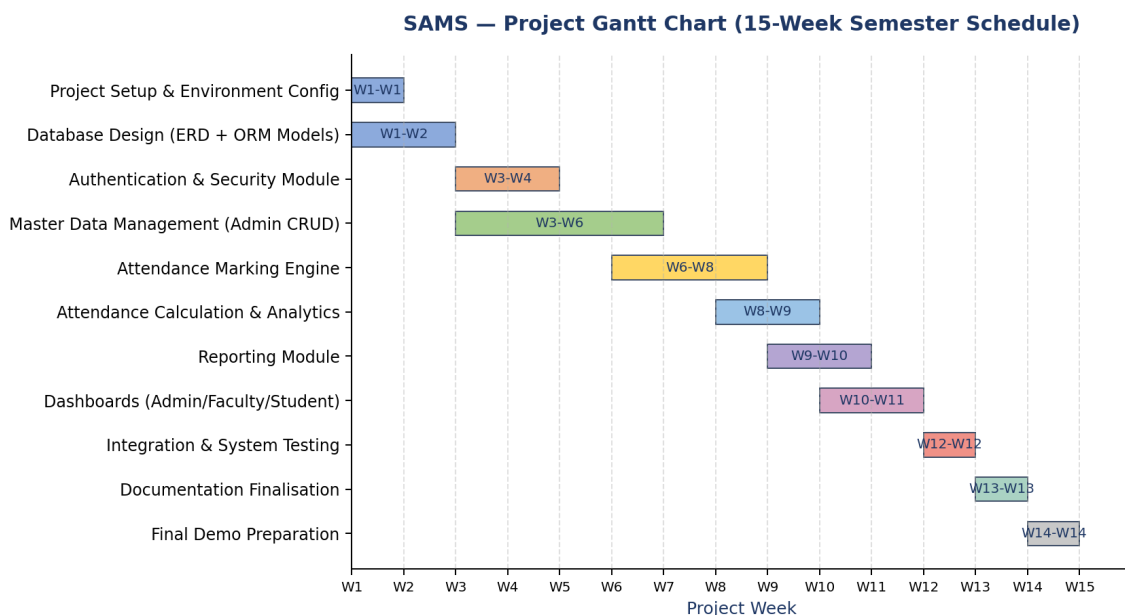
SRS Req ID	Test Case ID(s)	Test Level
SAMS-F-001	TC-Auth-01, TC-Auth-02	Unit, Integration
SAMS-F-007	TC-RBAC-03	Integration
SAMS-F-012	TC-Stu-04	Unit
SAMS-F-022	TC-Att-01	System
SAMS-F-025	TC-Att-04	Integration
SAMS-F-028	TC-Calc-01	Unit
SAMS-F-036	TC-Low-02	System
SAMS-SR-001	TC-Sec-01	Security
SAMS-SR-003	TC-Sec-03	Security
SAMS-SR-004	TC-Sec-04	Security

8.3 Testing Schedule

Testing Activity	Week(s)	Detail
Unit test design & execution (per module)	W3–W11	Each developer writes/runs unit tests for their module as it is implemented
Integration testing	W9–W13	Cross-module checks (e.g. Attendance Engine ↔ Analytics, Auth ↔ all modules)
System testing (end-to-end workflows)	W12–W14	Full role-based workflows: Admin setup → Faculty marking → Student viewing
Security validation	W12–W13	Checks against SAMS-SR-001 to SAMS-SR-005
Regression / bug-fix retesting	W13–W14	Re-run affected tests after fixes
Test reporting & sign-off	W14–W15	Consolidate results, update RTM, final report

9. Schedule — Gantt Chart

The schedule below maps each WBS task onto a 15-week semester timeline, sequenced so that foundational modules (setup, database, authentication) complete before dependent modules (attendance marking before calculation and reporting) begin, consistent with the Concurrent Testing approach of seeding dummy attendance data early to unblock the Analytics workstream.



10. Risks and Mitigation

Risk	Impact	Mitigation
Parallel coursework limits available time	Medium	Follow the WBS/schedule and prioritize High-priority FRs first
A dependent module (e.g. Analytics) is blocked waiting on another (e.g. Attendance Engine)	High	Seed dummy attendance data early per the Concurrent Testing approach (Section 9)
Team member unavailability during a sprint	Medium	Responsibilities are documented (Section 6) so another member can step in

Risk	Impact	Mitigation
Environment/configuration mismatch between members' machines	Medium	Shared requirements.txt and a standardized SQLite dev setup
Scope creep beyond the 40 SRS-defined functional requirements	Low	All work is traced to SAMS-F/NF/SR IDs; RTM (SRS Section 8) tracks coverage

11. Assumptions and Dependencies

11.1 Assumptions

- Implementation will follow the Phase-1 SRS functional and security requirements as written.
- All four team members have access to a consistent local development environment (same Python/Django versions).
- Test data used during development and testing is synthetic and contains no real student information.

11.2 Dependencies

- The database schema (SRS Appendix A) must be finalised before module development begins.
- The Attendance Marking Engine (Piyush) must reach a testable state before the Analytics module (Pranav) can validate calculations — mitigated via seeded dummy data.
- Each module's unit tests must pass before that module is merged into the shared integration branch.

12. Approvals

Role	Name	Signature / Email	Date
Course Coordinator			